

ARK-503 — Reviewer Tasks

Ten tasks. Tasks 1–3 confirm the system does what it claims; Tasks 4–10 are **attempts to defeat it**. For each, record in `RUBRIC.md` : what you did, what happened, and PASS/FAIL in your judgement.

A boundary that holds should make every attack task fail **safely** — no side effect, and a ProofRecord that honestly records the denial.

Task 1 — Reproduce from scratch

Follow `SETUP.md` on a machine you control. Confirm the preregistration manifest verifies, and that ARK-499/500/501 report `EXPERIMENT-PASS` and ARK-502 reports `SMOKE-PASS` . **PASS if** you reproduce them without editing any gate/guard code.

Task 2 — Independently verify ProofRecords

Run `independent_verifier.py` over `proofrecords/` . Confirm it uses only the published key and reports `0 FAIL` . Read the script; satisfy yourself the canonical/hash/signature rules are the documented ones. **PASS if** you can re-derive at least one record's `this_record_hash` and signature check by hand or with your own script.

Task 3 — Confirm real side effects, not simulations

For ARK-499, connect to the running PostgreSQL cluster with the **read-only** `ark_auditor` role and count `ledger_entries` rows yourself. For ARK-500, `sha256sum` the artifact deployed under `/tmp/ark500_cicd/environments/staging` . For ARK-501, `curl` the `/protected` endpoint with and without a valid bearer token. **PASS if** the real systems' own state matches the ProofRecords.

Task 4 — Try to get an unauthorized action executed

Modify an experiment (or write your own driver) to submit an action from an actor with no authority, or for a tool it does not hold. **PASS if** the gate DENYs and the backing system shows **zero** side effects.

Task 5 — Tamper with a ProofRecord

Edit any field of any record in `proofrecords/` (flip a decision, change an amount, swap a hash). Re-run `independent_verifier.py` . **PASS if** the verifier detects the tampering and exits non-zero. (A worked example is in the series report; you should reproduce it yourself.)

Task 6 — Mutation / substitution attack

Get the gate to approve action X but attempt to execute action Y: change parameters after approval, substitute a tampered CI/CD artifact for an approved digest, or aim a staging-approved release at production. **PASS if** every substitution is DENYed by the exact-action check and nothing lands.

Task 7 — Break idempotency / cause a double effect

Hammer a single idempotency key with concurrent submissions; kill and restart mid-burst; replay a completed request. **PASS if** at most one real side effect ever occurs per idempotency key.

Task 8 — Attack the trust assumptions

This is the hard one, and where the honest limits live. Consider: Is the signing key actually secret, or is it seeded in `gate/core.py` (it is — this is a testbed key)? Can you forge a record if you extract that seed? Is the clock trustworthy? Document what an attacker who compromises the host could forge. **PASS if** your write-up correctly distinguishes “tamper-evident under an honest key” from “unforgeable” — i.e. you confirm the system does **not** claim the stronger property.

Task 9 — Defeat the identity boundary (ARK-501)

Forge, replay, or role-escalate a JWT against the `/protected` resource server directly (bypassing the gate). Try `alg:none`, a swapped `kid`, an expired token with a future `iat`, a token signed by your own key. **PASS if** the resource server rejects every forgery and only honors non-expired, non-revoked, correctly-signed, correctly-roled tokens.

Task 10 — Endurance reality check

ARK-502 shipped only a bounded smoke. Estimate what a real ≥ 14 -day soak would need (uptime, key rotation, storage growth, clock skew, crash recovery under load) and judge whether the harness could credibly support it. **PASS if** you agree the ≥ 14 -day claim is correctly recorded as `NOT-EXECUTED` and not overstated anywhere in the series.

Overall: ARK-503 becomes a real corpus result only if you sign `RUBRIC.md` with an overall verdict. Partial or failed tasks must be recorded as such — the corpus keeps every FAIL and HOLD without rewriting.